

Generalizable Graph-level Anomaly Detection via Prompted Anomaly Expansion and Normality Extraction

Ge Zhang
School of Information and
Intelligent Science,
Donghua University
Shanghai, China
gezhang@dhu.edu.cn

Jiapei Chen
School of Information and
Intelligent Science,
Donghua University
Shanghai, China
2242808@mail.dhu.edu.cn

Guohao Sun*
School of Information and
Intelligent Science,
Donghua University
Shanghai, China
gshun@dhu.edu.cn

Xiu Fang
School of Information and
Intelligent Science,
Donghua University
Shanghai, China
xiu.fang@dhu.edu.cn

Zhenyu Yang
School of Computing,
Macquarie University
Sydney, Australia
zhenyu.yang@mq.edu.au

Xixun Lin*
Institute of Information
Engineering, Chinese
Academy of Sciences
Beijing, China
linxixun@iie.ac.cn

Liang Yang
School of Artificial
Intelligence, Hebei
University of Technology
Tianjin, China
yangliang@vip.qq.com

Abstract

Anomalous graphs, representing rare but critical deviations from normal graphs, frequently arise in high-stakes domains such as malicious website detection. Detecting them is highly challenging due to two key issues: (i) labeled anomalous graphs are extremely limited and fail to capture the diversity of real-world abnormality, restricting detection models from generalizing to unseen anomalies encountered in the open world, and (ii) normal graphs often contain spurious or atypical substructures that do not indicate anomalies but may induce models to misclassify normal variations as anomalies. To tackle these challenges, we propose G-GLAD, a generalizable graph-level anomaly detection framework. G-GLAD introduces two key innovations: (1) prompt-based anomaly space expansion, which injects learnable prompts into the graph representation process of known anomalies to simulate diverse unseen anomalous variants. This allows the model to learn a richer and more generalizable anomaly space; and (2) subgraph-based normality extraction, guided by the Information Bottleneck Principle, which isolates essential substructures for normality prediction while filtering out spurious motifs, improving robustness against structural noise. We conduct extensive experiments on ten real-world graph datasets under different empirical settings. The results demonstrate that G-GLAD achieves superior performance and generalizability in identifying anomalous graphs.

CCS Concepts

• Information systems → Data mining; • Computing methodologies → Neural networks.

*Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
WWW '26, Dubai, United Arab Emirates.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792429>

Keywords

Graph-level Anomaly Detection, Graph Prompt Learning, Graph Neural Networks

ACM Reference Format:

Ge Zhang, Jiapei Chen, Guohao Sun, Xiu Fang, Zhenyu Yang, Xixun Lin, and Liang Yang. 2026. Generalizable Graph-level Anomaly Detection via Prompted Anomaly Expansion and Normality Extraction. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792429>

1 Introduction

Graph-level Anomaly Detection (GLAD) presents a critical challenge in machine learning: identifying anomalous graphs that deviate from the majority without a comprehensive prior of what “anomalous” entails [23, 27]. The manifestations of anomalies in graphs are inherently diverse, arising from variations in both attributes and structure as well as their complex interactions [47]. For instance, web traffic sessions modeled as request graphs may exhibit suspicious attribute behaviors, while websites can be represented as graphs of interconnected pages, where malicious sites often reveal irregular linking patterns [26]. These examples underscore that anomalies can emerge through different relational or semantic irregularities, making it difficult to predefine what constitutes abnormality. Graph Neural Networks (GNNs) [9, 25, 37, 39, 49], with the capacity to jointly model attribute and structural dependencies, have thus become the *de facto* framework for GLAD.

Existing GNN-based GLAD methods can generally be divided into two paradigms. The first trains exclusively on labeled normal graphs, assigning anomaly scores to test graphs according to their deviation from learned normal patterns [22, 28, 42, 46]. Although effective in identifying obvious deviations, these models struggle to identify subtle or structurally complex anomalies, as their decision boundaries are determined only by normal graphs. In addition, tuning the anomaly score threshold for each dataset reduces the practicality of the model in real-world scenarios. The second paradigm formulates GLAD as an imbalanced binary classification

problem [7, 17, 47]. Even a small fraction of labeled anomalous graphs can provide crucial information about anomalous structures and features that normal-only training would miss; however, the limited diversity of labeled anomalies, due to the inherent selection bias in the data, leaves the model blind to many abnormal manifestations not seen during training. This fundamentally limits its generalization to previously unseen anomaly patterns.

Two fundamental challenges in GLAD deserve attention. **Challenge 1 – Identifying anomalies in the open world when only limited labeled data are available.** Although a few anomalous graphs are labeled, they rarely capture the full range of abnormal patterns encountered in the open world. Expanding the model’s understanding about anomalies beyond the limited labeled set is crucial; a promising direction is to use these known anomalies as seeds to generate more anomalous variants. But this remains non-trivial, as general-purpose generative models like Large Language Models (LLMs) [10, 16] struggle with complex graph structures; this motivates a specialized GNN-compatible method capable of capturing diverse deviations. **Challenge 2 – Modeling normal graphs with structural noise.** Normal graphs often contain spurious or atypical substructures that do not indicate true anomalies. These irregularities can mislead models into classifying normal variations as anomalous, thereby weakening anomaly characterization [21, 32, 35]. Robust GLAD models must distinguish meaningful deviations from structural noise to ensure reliable detection.

In this paper, we propose Generalizable Graph-Level Anomaly Detection (G-GLAD). The core idea of G-GLAD is that effective generalization hinges on exposing the model to diverse anomalous patterns while ensuring stable representations of normality. G-GLAD introduces three core components, i.e., the Graph Anomaly Prompter, the Subgraph-based Normality Extractor and the Predictor. Graph prompting employs learnable prompt vectors to modulate node or graph representations, thereby adapting GNNs to various downstream tasks. This modulation mechanism can create variations in graph features and structures, which can be leveraged to simulate potential anomalous patterns. Therefore, the Graph Anomaly Prompter in G-GLAD injects feature-level and structure-level deviations through prompt vectors during the GNN message-passing process, enabling the model to generalize beyond a small number of labeled anomalies. The Subgraph-based Normality Extractor identifies key subgraphs from normal graphs that preserve essential normal patterns while filtering out noise, producing stable representations of normality. The Predictor builds the decision boundaries that jointly consider normal, real anomalous, and prompted anomalous variants. By explicitly expanding the anomaly space and stabilizing normality, G-GLAD can distinguish anomalous graphs from normal ones and generalize to previously unseen anomalies. Our contributions can be summarized as follows:

- **Problem.** We identify a critical problem: the limited labeled anomalies and the prevalence of spurious substructures in normal graphs severely hinder the effectiveness and generalization of existing GLAD models.
- **Algorithm.** We propose G-GLAD to build discriminative and generalizable decision boundaries. G-GLAD enlarges the anomaly space by a Graph Anomaly Prompter and stabilizes normal representations by a Subgraph-based Normality

Extractor. Various prompted anomaly variants expand the model’s understanding of anomalies beyond labeled samples.

- **Evaluation.** Extensive evaluations on multiple real-world graph datasets show that G-GLAD consistently outperforms baselines in identifying anomalous graphs and demonstrate superior generalization to previously unseen anomalies.

2 Related Work

2.1 Graph-level Anomaly Detection

GLAD [42, 45] is harder than detecting anomalies within a single graph (e.g. abnormal nodes [4, 5, 19, 40, 44], edges [14], or subgraphs [1]) [15, 31], because it requires analyzing multiple graphs, each with irregular and complex structures [41]. Early methods used Graph Kernels (GKs) [2, 12, 13, 30] to measure graph similarity and flag outliers. Recently, GNNs [11, 29, 34, 36, 39] have become the dominant approach for learning from graph-structured data. GNN-based GLAD can be broadly categorized into two groups.

Learning from Normal Graphs. The first class of GLAD methods uses only labeled normal graphs [3, 6]. For example, OCGIN [48] maps normals to a hypersphere center and computes anomaly scores as the distance to the center. OCGTL [28] trains multiple GNNs jointly via transformation learning to model normal distributions. GLocalKD [22] trains a GNN to replicate the predictions of a fixed, randomly initialized GNN to extract dominant normal patterns. SIGNET [18] learns multiple views of normal graphs via cross-view contrastive learning. However, relying only on normals to define decision boundaries leaves the model without anomalous guidance, limiting boundary expressiveness.

Learning from Classifiers. The second class of GLAD methods cast detection as imbalanced binary graph classification. For example, iGAD [44] integrates an attribute-aware graph convolution with a substructure-aware deep Random Walk Kernel to capture anomalous attributes and substructures. RQGNN [7] models the Rayleigh Quotient (RQ) and uses a Chebyshev Wavelet GNN with RQ-pooling to extract discriminative spectral signals, reflecting differences in spectral energy between normal and anomalous graphs. UniGAD [17] applies MRQSampler to select subgraphs with maximal spectral energy, preserving critical anomalous patterns and improving discrimination. Despite their progress, they depend on few labeled anomalies, limiting anomaly-space coverage. To mitigate the limited generalization of GLAD, SDM [3] proposes to generate pseudo-anomalies by perturbing normal graphs. However, its generation adheres to the homophily assumption, i.e., similar nodes should be connected, which may limit the diversity of generated anomalies and hinder generalization to unseen anomalies.

2.2 Graph Prompt Learning

Graph prompt learning [33] aims to adapt pre-trained GNNs [38] to downstream tasks by introducing learnable prompts into GNNs, bridging the gap between pre-training and downstream tasks. For example, GraphPrompt [20] introduces learnable prompts by parameterizing the readout operation, which adaptively guides subgraph aggregation and enables task-aware utilization of pre-trained models. GPF [8] learns and injects a universal “prompt feature” into node features to align downstream tasks with diverse pre-training strategies. MultiGPrompt [43] inserts prompt vectors into node

representations at each hidden layer, where a dual-prompt mechanism combines composed prompts for task-specific knowledge and open prompts for global knowledge across multiple pretext tasks. EdgePrompt [9] extends prompts to graph structures to optimize graph representations for downstream tasks. Inspired by these methods, we leverage prompt vectors to expand the anomaly space. This allows us to transform a limited set of known anomalies into a diverse spectrum of anomalies.

3 Preliminary

Graph. Let $G = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ denote an undirected graph, where $\mathcal{V}_G$ and $\mathcal{E}_G$ are the sets of n nodes and m edges, respectively. The adjacency matrix is $\mathbf{A} \in \mathbb{R}^{n \times n}$, where $\mathbf{A}_{ij} = 1$ if edge $(v_i, v_j) \in \mathcal{E}_G$ and $\mathbf{A}_{ij} = 0$ otherwise. For a node $v_i \in \mathcal{V}_G$, let $\mathcal{N}(v_i)$ denote its one-hop neighborhood. The node feature matrix is $\mathbf{X} \in \mathbb{R}^{n \times d}$, where the i -th row $\mathbf{x}_i$ represents the feature vector of the node v_i .

Problem Definition. We denote the training dataset as $\mathcal{D}_{tr} = \{(G_i, y_i)\}_{i=1}^N$, where $y_i \in \{0, 1\}$, $y_i = 0$ if G_i is a normal graph $G^n \sim \mathcal{P}_n$, and $y_i = 1$ if G_i is an anomalous graph $G^a \sim \mathcal{Q}$. In this paper, G^n and G^a refer to any labeled normal and anomalous graphs used for training, respectively. In practice, the dataset is highly imbalanced, i.e., $N_{\text{normal}} \gg N_{\text{anomaly}}$. To reflect the diversity of anomalies, we assume that anomalous graphs originate from a mixture of distributions $\mathcal{Q} = \{\mathcal{Q}_k \mid k = 1, 2, \dots, K\}$, where each $\mathcal{Q}_k$ corresponds to a distinct anomaly pattern that causes a graph to be anomalous. During training, only a limited subset $\mathcal{Q}_{\text{train}} \subset \mathcal{Q}$ is observed and labeled, forming the training anomaly space. At test time, the model encounters a test set $\mathcal{D}_{te} = \{G_j\}_{j=1}^M$, where some anomalous graphs may originate from previously unseen distributions $\mathcal{Q}_k \in \mathcal{Q} \setminus \mathcal{Q}_{\text{train}}$. Consequently, GLAD goes beyond a standard imbalanced binary classification problem, as the anomaly space is open and partially observable. The model must therefore capture the distribution of normal graphs $\mathcal{P}_n$ and generalize to anomalous graphs G_{new}^a originating from unseen distributions.

Graph Convolution. The l -th graph convolution layer of a GNN with L layers can be written as:

$$\mathbf{h}_{v_i}^{(l)} = \text{COMB}\left(\mathbf{h}_{v_i}^{(l-1)}, \text{AGG}(\{\mathbf{h}_{v_j}^{(l-1)} \mid v_j \in \mathcal{N}(v_i)\})\right), \quad (1)$$

where $\mathbf{h}_{v_i}^{(l)} \in \mathbb{R}^{d'}$ denotes the representation of node v_i at the l -th convolution layer ($l \in \{1, 2, \dots, L\}$), with initial node representation $\mathbf{h}_{v_i}^{(0)} = \mathbf{x}_i$. Here, $\text{AGG}(\cdot)$ aggregates information from the neighbors of v_i , and $\text{COMB}(\cdot)$ combines the aggregated neighborhood with the node's representation from the previous layer.

4 Methodology

In this section, we discuss how to achieve generalizable graph-level anomaly detection using the proposed G-GLAD. As shown in Figure 1, G-GLAD integrates anomaly space expansion, normality extraction, and prediction into a unified framework. The Graph Anomaly Prompter (Section 4.1) learns feature- and structure-level anomaly prompts and inserts them into a shared GNN-based Predictor, allowing the generation of diverse prompted anomalous representation variants. This expands the anomaly space beyond limited labeled anomaly examples. The Subgraph-based Normality Extractor (Section 4.2) follows the information bottleneck principle to extract

compact subgraphs that preserve key normality while removing noise. Finally, the Predictor (Section 4.3) uses these enriched anomaly cues and extracted normal subgraphs to learn discriminative decision boundaries that can be generalized to unseen anomalies. The time complexity analysis is provided in Section A.4.3.

4.1 Graph Anomaly Prompter

To expand the known anomaly space beyond the labeled anomalous graphs, we propose the Graph Anomaly Prompter $\phi(\cdot)$, which provides learnable anomaly prompt vectors to transform known anomalies G^a into diverse anomaly variants covering feature-level, structure-level, and combined deviations. $\phi(\cdot)$ does not modify graph features or structures directly. Instead, it injects these anomaly prompt vectors into the GNN message-passing process, operating in the representation space to produce prompted anomalous graph representation variants. By generating these prompted variants, the model explores previously unseen regions of the anomaly space, reduces bias toward known anomalies, and improves generalization to novel anomaly patterns. Next, we describe how to generate new anomaly patterns by introducing feature-level and structure-level anomaly prompts, enabling the model to expand the anomaly space along both feature and structural dimensions.

4.1.1 Feature-level Anomaly Prompt. During the forward pass of the GNN, feature-level anomaly prompts are realized by adding the prompt vector to immediate node representations of known anomalies at each graph convolution layer. Formally, for node $v_i \in G^a$, at the layer l ,

$$\hat{\mathbf{h}}_{v_i}^{(l)} = \text{COMB}\left(\hat{\mathbf{h}}_{v_i}^{(l-1)} + \mathbf{u}_{v_i}^{(l)}, \text{AGG}(\{\hat{\mathbf{h}}_{v_j}^{(l-1)} \mid v_j \in \mathcal{N}(v_i)\})\right), \quad (2)$$

where $\mathbf{u}_{v_i}^{(l)} \in \mathbb{R}^{d'}$ is the feature-level anomaly prompt vector for node v_i at the l -th layer of the GNN architecture. By applying the graph pooling operation over $\{\hat{\mathbf{h}}_{v_i}^{(l)}, \forall v_i \in \mathcal{V}_{G^a}\}$, we obtain the prompted anomalous graph representation variant $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}$, i.e.,

$$\tilde{\mathbf{h}}_{G^a}^{\text{feature}} = \text{POOLING}\left(\{\hat{\mathbf{h}}_{v_i}^{(l)} \mid v_i \in \mathcal{V}_{G^a}\}\right), \quad (3)$$

which serves as a variant of $\mathbf{h}_{G^a}$, capturing anomaly potentials induced by feature-level deviations on G^a .

However, directly assigning an independent learnable prompt vector $\mathbf{u}_{v_i}^{(l)}$ to each node v_i at every GNN layer may greatly increase the number of parameters, potentially leading to unstable node representations and more difficult optimization. To mitigate this, we instead parameterize $\mathbf{u}_{v_i}^{(l)}$ as a weighted combination of K shared anchor prompt vectors $\{\mathbf{t}_1^{(l)}, \mathbf{t}_2^{(l)}, \dots, \mathbf{t}_K^{(l)}\}$, which effectively constrains the space of possible prompts and stabilizes learning. Formally, at the l -th layer, the feature-level prompt vector for node v_i can be computed as

$$\mathbf{u}_{v_i}^{(l)} = \sum_{k=1}^K b_k^{(l)}(v_i) \cdot \mathbf{t}_k^{(l)}, \quad (4)$$

where $b_k^{(l)}(v_i)$ denotes the k -th entry of the weight vector $\mathbf{b}^{(l)}(v_i)$. Because all nodes at the l -th layer share the same set of anchor prompts, the weight vector $\mathbf{b}^{(l)}(v_i)$ plays a crucial role in distinguishing node v_i from the others. Designing a reliable mechanism to

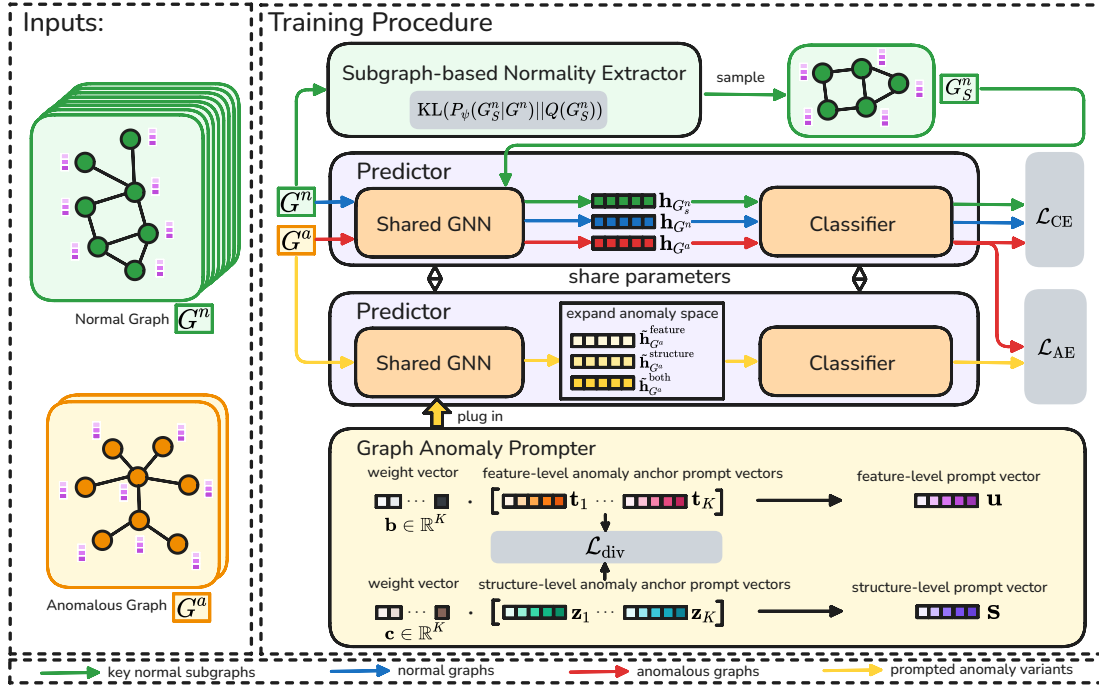


Figure 1: Illustration of the proposed G-GLAD architecture. Green, blue, red, and yellow arrows indicate the learning processes for key normal subgraphs, labeled normal graphs, labeled anomalous graphs, and prompted anomaly variants, respectively.

produce $\mathbf{b}^{(l)}(v_i)$ is therefore essential. We compute $\mathbf{b}^{(l)}(v_i)$ using a scoring function $\text{Score}_{\text{feature}}^{(l)}(\cdot) : \mathbb{R}^{d'} \rightarrow \mathbb{R}^K$, defined as

$$\mathbf{b}^{(l)}(v_i) = \text{SOFTMAX} \left(\text{Score}_{\text{feature}}^{(l)}(\hat{\mathbf{h}}_{v_i}^{(l-1)}) \right), \quad (5)$$

where $\text{SOFTMAX}(\cdot)$ is the softmax operation, and $\text{Score}_{\text{feature}}^{(l)}(\cdot)$ is an Multi-Layer Perceptron (MLP) mapping the node's previous-layer representation to the corresponding anchor weights.

4.1.2 Structure-level Anomaly Prompt. Since GNNs learn graphs by aggregating information from neighboring nodes, the embeddings inherently encode local graph structure. To generate potential structure-level anomalies, we inject anomaly prompt vectors into the representations of neighboring nodes before aggregation, to create potential structural deviations in the representation space without modifying the original graph structure. Formally, for node $v_i \in G^a$, at the layer l ,

$$\mathbf{h}'_{v_i}{}^{(l)} = \text{COMB} \left(\mathbf{h}'_{v_i}{}^{(l-1)}, \text{AGG}(\{\mathbf{h}'_{v_j}{}^{(l-1)} + \mathbf{s}_{v_i, v_j}^{(l)} \mid v_j \in \mathcal{N}(v_i)\}) \right), \quad (6)$$

where $\mathbf{s}_{v_i, v_j}^{(l)} \in \mathbb{R}^{d'}$ is the structural-level anomaly prompt vector for the edge $(v_i, v_j) \in \mathcal{E}_{G^a}$. Thus, the prompted anomalous graph representation $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$, which encodes the anomaly potentials induced by structural-level perturbations on G^a , is obtained by applying pooling over $\{\mathbf{h}'_{v_i}{}^{(l)} \mid v_i \in \mathcal{V}_{G^a}\}$, i.e.,

$$\tilde{\mathbf{h}}_{G^a}^{\text{structure}} = \text{POOLING} \left(\{\mathbf{h}'_{v_i}{}^{(l)} \mid v_i \in \mathcal{V}_{G^a}\} \right). \quad (7)$$

Similar to the feature-level anomaly prompt, the structure-level anomaly prompt $\mathbf{s}_{v_i, v_j}^{(l)}$ is also formulated as a weighted combination of multiple anchor prompts. The set of K structure-level anchor prompts are denoted as $\{\mathbf{z}_1^{(l)}, \mathbf{z}_2^{(l)}, \dots, \mathbf{z}_K^{(l)}\}$. Formally, at the l -th layer, the structure-level prompt vector $\mathbf{s}_{v_i, v_j}^{(l)}$ can be computed as

$$\mathbf{s}_{v_i, v_j}^{(l)} = \sum_{k=1}^K c_k^{(l)}(v_i, v_j) \cdot \mathbf{z}_k^{(l)}, \quad (8)$$

where $c_k^{(l)}(v_i, v_j)$ is the weight of the anchor prompt vector $\mathbf{z}_k^{(l)}$. Specifically, the elements $c_1^{(l)}, \dots, c_K^{(l)}$ of vector $\mathbf{c}^{(l)}(v_i, v_j) \in \mathbb{R}^K$ denote the weights of anchor prompts $\mathbf{z}_1^{(l)}, \dots, \mathbf{z}_K^{(l)}$, respectively, for edge $(v_i, v_j) \in \mathcal{E}_{G^a}$. We define a score function $\text{Score}_{\text{structure}}^{(l)}(\cdot) : \mathbb{R}^{d'} \rightarrow \mathbb{R}^K$ to obtain the weight vector $\mathbf{c}^{(l)}(v_i, v_j)$. Formally,

$$\mathbf{c}^{(l)}(v_i, v_j) = \text{SOFTMAX} \left(\text{Score}_{\text{structure}}^{(l)}(\mathbf{h}'_{v_i}{}^{(l-1)}, \mathbf{h}'_{v_j}{}^{(l-1)}) \right). \quad (9)$$

Notably, the score function $\text{Score}_{\text{structure}}^{(l)}(\cdot)$ takes the pair of nodes v_i and v_j as the input to learn the relationship between them in the structure-level anomaly potential $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$. We realize the function $\text{Score}_{\text{structure}}^{(l)}(\cdot)$ through the MLP layers. Formally,

$$\text{Score}_{\text{structure}}^{(l)}(\mathbf{h}'_{v_i}{}^{(l-1)}, \mathbf{h}'_{v_j}{}^{(l-1)}) = \text{MLP} \left(\text{CAT}(\mathbf{h}'_{v_i}{}^{(l-1)}, \mathbf{h}'_{v_j}{}^{(l-1)}) \right), \quad (10)$$

where $\text{CAT}(\cdot, \cdot)$ denotes the vector concatenation operation.

4.1.3 Anomaly Expansion. In addition to $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}$ and $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$, we further obtain another type of prompted anomalous graph representation $\tilde{\mathbf{h}}_{G^a}^{\text{both}}$ by simultaneously incorporating feature-level and structure-level anomaly prompts into the GNN message passing process. Formally,

$$\tilde{\mathbf{h}}_{G^a}^{\text{both}} = \text{POOLING}(\tilde{\mathbf{h}}_{v_i}^{(l)} \mid v_i \in \mathcal{V}_{G^a}), \quad (11)$$

where the node representation is computed as

$$\tilde{\mathbf{h}}_{v_i}^{(l)} = \text{COMB}(\tilde{\mathbf{h}}_{v_i}^{(l-1)} + \mathbf{u}_{v_i}^{(l)}, \text{AGG}(\tilde{\mathbf{h}}_{v_j}^{(l-1)} + \mathbf{s}_{v_i, v_j}^{(l)} \mid v_j \in \mathcal{N}(v_i))). \quad (12)$$

Hence, under such design, each labeled anomalous graph G^a is associated with three distinct prompted anomalous representation variants, including $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}$, $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$, and $\tilde{\mathbf{h}}_{G^a}^{\text{both}}$, which collectively enrich the anomaly space.

4.2 Subgraph-based Normality Extractor

The Subgraph-based Normality Extractor $\psi(\cdot)$ is designed to remove spurious or atypical motifs from normal graphs, yielding the key subgraph G_S^n that preserves only the information essential for correct identification as normal, i.e., $G_S^n = \psi(G^n)$. Formally, based on the Information Bottleneck Principle [24], the optimization objective can be formulated as

$$\min_{\psi(\cdot)} I(G_S^n; G^n) - I(G_S^n; y), \quad \text{s.t. } G_S^n = \psi(G^n), \quad y = 0, \quad (13)$$

where $y = 0$ denotes the label of normal graphs. This objective simultaneously compresses the input graph by minimizing the mutual information $I(G_S^n; G^n)$, while retaining label-relevant information by maximizing the mutual information $I(G_S^n; y)$.

On the one hand, to maximize the mutual information between the extracted subgraph and the label, i.e., $I(G_S^n; y)$, we need to find its lower bound. Formally,

$$I(G_S^n; y) \geq \mathbb{E}_{G_S^n} [\log P(y = 0 | G_S^n)] + H(y = 0), \quad (14)$$

where $H(y = 0)$ is the entropy of y and thus a constant. Ignoring it, we can obtain

$$I(G_S^n; y) \geq \mathbb{E}_{G_S^n} [\log P(y = 0 | G_S^n)]. \quad (15)$$

The posterior $\log P(y = 0 | G_S^n)$ can be approximated by the predictor $f(\cdot)$ that discriminates between normal and anomalous graphs.

On the other hand, to minimize the mutual information between the normal graph and the subgraphs extracted from it, i.e., $I(G_S^n; G^n)$, we adopt a variational information bottleneck approach to obtain an upper bound

$$I(G_S^n; G^n) \leq \mathbb{E}_{G^n} [\text{KL}(P_\psi(G_S^n | G^n) || Q(G_S^n))], \quad (16)$$

where $Q(G_S^n)$ is a variational approximation of the marginal distribution $P(G_S^n)$, and $\text{KL}(\cdot || \cdot)$ denotes the Kullback–Leibler divergence. Detailed derivation of tractable variational bounds for Eq. (15) and Eq. (16) is given in Appendix A.2.

Next, we describe the procedure for extracting the subgraph G_S^n from a normal graph G^n and for modeling the KL-divergence between $P_\psi(G_S^n | G)$ and $Q(G_S^n)$. The Subgraph-based Normality Extractor $\psi(\cdot)$ encodes the input normal graph G^n using a GNN, yielding a set of node representations $\{\mathbf{h}_{v_i} \mid v_i \in \mathcal{V}_{G^n}\}$. For each edge $(v_i, v_j) \in G^n$, an MLP estimates its contribution to the overall normality of G^n . The resulting score $p(v_i, v_j)$ is mapped to $[0, 1]$

and interpreted as the probability of including the edge between v_i and v_j in the extracted subgraph G_S^n . Subsequently, the optimized subgraph $\hat{G}_S^n$ is sampled from G^n according to a Bernoulli distribution, i.e., $(v_i, v_j)' \sim \text{Bernoulli}(p(v_i, v_j))$. Hence,

$$P_\psi(G_S^n | G^n) = \prod_{(v_i, v_j) \in \mathcal{E}_{G^n}} P((v_i, v_j)' | p(v_i, v_j)). \quad (17)$$

In parallel, the prior distribution $Q(G_S^n)$ is defined as a Bernoulli distribution with a hyper-parameter o , $o \in [0, 1]$. Accordingly, the KL-divergence $\text{KL}(P_\psi(G_S^n | G^n) || Q(G_S^n))$ measures the discrepancy between these two Bernoulli distributions. Formally,

$$\begin{aligned} \text{KL}(P_\psi(G_S^n | G^n) || Q(G_S^n)) = \\ \sum_{(v_i, v_j) \in \mathcal{E}_{G^n}} p(v_i, v_j) \log \frac{p(v_i, v_j)}{o} + (1 - p(v_i, v_j)) \log \frac{1 - p(v_i, v_j)}{1 - o}. \end{aligned} \quad (18)$$

4.3 Predictor

The Predictor $f(\cdot)$ is implemented as a shared GNN comprising graph convolution and pooling layers, followed by a classifier that discriminates anomalous graphs from normal ones.

While anomalous graphs G^a , normal graphs G^n , and the key subgraphs G_S^n can be directly encoded into embeddings $\mathbf{h}_{G^a}$, $\mathbf{h}_{G^n}$, and $\mathbf{h}_{G_S^n}$ through the shared GNN backbone, the prompted anomalous variants exist only in the representation space without explicit graph structures or features to process. To resolve this mismatch, we design the Graph Anomaly Prompter as a pluggable module that injects learnable feature- and structure-level prompts into the message-passing process. In this way, the Predictor is guided to generate diverse anomalous representation variants— $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}$, $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$, and $\tilde{\mathbf{h}}_{G^a}^{\text{both}}$ —expanding the anomaly space beyond the limited labeled graphs. Afterwards, Predictor is thus optimized not only to capture graph normality but also to discriminate across a broader and more diverse range of anomalies. Training is guided by three complementary objectives: a cross-entropy loss for supervised discrimination, an anomaly expansion loss for enlarging anomaly space, and a diversity loss for encouraging varied anchor prompt vectors.

4.3.1 Cross-entropy Loss Function. The cross-entropy loss is applied to three types of graph representations— $\mathbf{h}_{G^n}$, $\mathbf{h}_{G^a}$, and $\mathbf{h}_{G_S^n}$ —to perform classification. Formally,

$$\mathcal{L}_{\text{CE}} = \text{CE}(y, f(\mathbf{h}_G)), \quad (19)$$

where $f(\mathbf{h}_G)$ denotes the classification logits computed from $\mathbf{h}_G$, which correspond to $\mathbf{h}_{G^n}$, $\mathbf{h}_{G^a}$, or $\mathbf{h}_{G_S^n}$. Here, $y \in \{0, 1\}$ is the ground-truth label. In particular, the maximization of $\mathbb{E}_{G_S^n} [\log P(y = 0 | G_S^n)]$ (as described in Equation (15)) is realized by applying the cross-entropy loss to the key subgraph representation $\mathbf{h}_{G_S^n}$.

4.3.2 Anomaly Expansion Loss Function. Minimizing the standard cross-entropy loss (e.g., $\text{CE}(y = 1, f(\tilde{\mathbf{h}}_{G^a}))$) pushes the predicted anomaly-class logit of an anomalous input toward nearly 100%, producing overconfident predictions. Here, $\tilde{\mathbf{h}}_{G^a}$ denotes prompted anomaly variants derived from a labeled anomaly G^a , including $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}$, $\tilde{\mathbf{h}}_{G^a}^{\text{structure}}$, and $\tilde{\mathbf{h}}_{G^a}^{\text{both}}$. Such rigid enforcement may narrow the decision boundary and cause the model to overlook diverse anomaly patterns. To encourage exploration of a broader anomaly

space, we propose the anomaly expansion loss, which relaxes the confidence requirement on variants by requiring only prediction-label consistency with their source anomalies. Formally,

$$\mathcal{L}_{AE} = \text{CE}(\text{argmax}f(\tilde{\mathbf{h}}_{G^a}), f(\mathbf{h}_{G^a})), \quad (20)$$

where $\text{argmax}f(\tilde{\mathbf{h}}_{G^a})$ denotes the predicted label on $\tilde{\mathbf{h}}_{G^a}$. This design pushes the anomaly-class logits of the real anomalies toward 100%, while relaxing the requirement on the logits of the prompted anomalous variants. In other words, the real anomalies act as high-confidence anchors, preventing the model from collapsing all variants to their source anomalies and enabling exploration of a broader anomaly space.

4.3.3 Diversity Loss Function. In the Graph Anomaly Prompter, at each graph convolution layer l , we introduce shared anchor prompt vectors $\{\mathbf{t}_1^{(l)}, \mathbf{t}_2^{(l)}, \dots, \mathbf{t}_K^{(l)}\}$ and $\{\mathbf{z}_1^{(l)}, \mathbf{z}_2^{(l)}, \dots, \mathbf{z}_K^{(l)}\}$ for generating feature- and structure-level anomaly prompts, respectively. To encourage the diversity among prompted anomalous variants, we impose a decorrelation constraint on anchor prompt vectors. This constraint drives them to be approximately orthogonal in the representation space, preventing redundancy and improving coverage of different anomaly patterns. Concretely, we concatenate $\{\mathbf{t}_1^{(l)}, \mathbf{t}_2^{(l)}, \dots, \mathbf{t}_K^{(l)}\}$ and $\{\mathbf{z}_1^{(l)}, \mathbf{z}_2^{(l)}, \dots, \mathbf{z}_K^{(l)}\}$ into matrices $\mathbf{T}^{(l)} \in \mathbb{R}^{K \times d'}$ and $\mathbf{Z}^{(l)} \in \mathbb{R}^{K \times d'}$, respectively. Then we calculate the Gram matrices $\mathbf{W}_{\text{feature}}^{(l)}$ and $\mathbf{W}_{\text{structure}}^{(l)}$ of these two matrices separately,

$$\mathbf{W}_{\text{feature}}^{(l)} = \mathbf{T}^{(l)} \mathbf{T}^{(l)\top}, \quad \mathbf{W}_{\text{structure}}^{(l)} = \mathbf{Z}^{(l)} \mathbf{Z}^{(l)\top}. \quad (21)$$

Further, we introduce the following diversity loss:

$$\mathcal{L}_{\text{DIV}} = \sum_{l=1}^L \left\| \mathbf{W}_{\text{feature}}^{(l)} - \mathbf{I}_{\text{feature}} \right\|_F^2 + \left\| \mathbf{W}_{\text{structure}}^{(l)} - \mathbf{I}_{\text{structure}} \right\|_F^2, \quad (22)$$

where $\mathbf{I}_{\text{feature}}, \mathbf{I}_{\text{structure}} \in \mathbb{R}^{K \times d'}$ denote the identity matrices.

Overall, the final loss function of G-GLAD can be defined as:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{KL} \text{KL}(P_\psi(G_S^n | G^n) || Q(G_S^n)) + \lambda_{AE} \mathcal{L}_{AE} + \lambda_{\text{DIV}} \mathcal{L}_{\text{DIV}}, \quad (23)$$

where λ_{KL} , λ_{AE} , and λ_{DIV} are weighting coefficients. The pseudo-code of G-GLAD is presented in Algorithm 1 (Appendix A.1).

5 Experiments

5.1 Experimental Setting

5.1.1 Datasets. We conduct experiments on ten real-world datasets, all sourced from TUDataset¹. The real-world datasets cover: social networks (IMDB-BINARY, IMDB-MULTI, REDDIT-BINARY, COLLAB), bioinformatics networks (PROTEINS_full, DD, ENZYMES), molecular networks (AIDS, COX2) and computer vision dataset (MSRC_9). These datasets were originally provided for graph classification tasks. Following standard practice, we treat class 0 as the normal class and all other classes as anomalous.

For **binary datasets** (IMDB-BINARY, PROTEINS_full, REDDIT-BINARY, DD, AIDS, and COX2), which have only two classes, the test set does not contain any unseen anomaly classes. For **multi-class datasets** (ENZYMES with 6 classes, COLLAB with 3, IMDB-MULTI with 3, and MSRC_9 with 8), we evaluate models under two

settings: (1) **Closed Anomaly-set Setting (CAS)**. All anomalous classes (i.e., all classes except class 0) are included during training. Testing is performed on all classes as well, corresponding to the closed-world assumption; (2) **Open Anomaly-set Setting (OAS)**. During training, the model sees only class 1 as anomalous (all other classes are excluded except for the normal class). At test time, all remaining classes (excluding class 0 and class 1) are treated as unseen anomalies, simulating unknown anomaly patterns in an open-world scenario. More details are provided in Appendix A.3.1.

5.1.2 Baselines. We compare G-GLAD with the following three types of competitors: (1) **Learning from Normal Graphs**. Methods like OCGIN [48], OCGTL [28], GLocalKD [22], and SIGNET [18] are trained exclusively on normal graphs and inherently do not require distributions between CAS and OAS setting. (2) **Learning from Classifiers**. Approaches such as iGAD [47] and RQGNN [7], which formulate GLAD as an imbalanced binary classification task; (3) **Graph Prompt Learning**. Recent frameworks including GPF [8] and EdgePrompt [9], which can adapt pre-trained GNNs to the GLAD task via prompt-based mechanisms.

5.1.3 Implementation. Our experiments were conducted on a Linux server with an Intel(R) Xeon(R) Gold 5218R 20-core CPU, 64GB of RAM, and an NVIDIA RTX 4090 GPU with 24GB memory. In G-GLAD, we adopt a 2-layer GIN [39] as the GNN backbone. We set the number of anchor prompt vectors K to 10 at each GNN layer. For all datasets, 40% of the data are used for training, 20% for validation, and the remaining 40% for testing. To mimic the real-world imbalance between normal and anomalous cases, our paper evaluates G-GLAD, classifier-based and graph prompt learning baselines under two settings with normal-to-anomalous ratios of 15:1 and 7:1 in the training graph set. We adopt three metrics: AUC, AUPR, and Macro F1-score. Detailed descriptions of the metrics are provided in Appendix A.3.2. Each experiment is repeated 5 times with different random seeds, and we report the mean and standard deviation. Our code is available at <https://github.com/Lab821-gz/G-GLAD>.

5.2 Performance Comparison

To evaluate the effectiveness and generalization of G-GLAD, we conducted comparative experiments under both OAS and CAS settings. In the OAS setting, where the test set contains both seen and unseen anomalies, and Table 1 reports the AUC and AUPR results comparing G-GLAD with eight baseline methods. Results on the test set with seen anomalies removed are provided in Appendix A.4.1. In the CAS setting, Table 2 reports the AUC results, while the corresponding AUPR and Macro F1-score results are provided in Appendix A.4.2. Note that methods of the *Learning from Normal Graphs* type are trained exclusively on normal graphs. By definition, their evaluation involves only unseen anomalies during testing, so they are not separately evaluated under the CAS and OAS settings.

5.2.1 Performance on Open Anomaly-set Setting. We conduct a multi-anomalous-class GLAD experiment on ENZYMES, COLLAB, IMDB-MULTI, and MSRC_9 to evaluate the generalization of our methods under the OAS setting. From Table 1, it is clear that G-GLAD achieves the best performance. In contrast, GLocalKD and SIGNET achieve relatively low AUPR, with 21.07% and 23.55% on ENZYMES, and 22.41% and 21.48% on MSRC_9, suggesting that

¹<https://chrsmrrs.github.io/datasets/docs/datasets/>

Table 1: AUCs and AUPRs (% , mean $\pm$ std, 5 trials) of GLAD methods on dataset under OAS setting. Methods, including our method, classifier-based baselines, and graph prompt baselines, were trained with a 15:1 normal-to-anomalous graph ratio. The best results are marked in bold.

Dataset	ENZYMES		COLLAB		IMDB-MULTI		MSRC_9	
Method	AUC	AUPR	AUC	AUPR	AUC	AUPR	AUC	AUPR
OCGIN	58.04 $\pm$ 7.57	88.25 $\pm$ 3.01	70.94 $\pm$ 2.66	74.50 $\pm$ 2.20	47.72 $\pm$ 6.17	67.80 $\pm$ 4.62	93.67 $\pm$ 6.92	99.36 $\pm$ 0.72
OCGTL	57.20 $\pm$ 11.33	87.24 $\pm$ 4.85	69.70 $\pm$ 0.87	73.19 $\pm$ 0.73	47.29 $\pm$ 6.88	68.32 $\pm$ 5.15	94.17 $\pm$ 2.36	99.55 $\pm$ 0.13
GLocalKD	54.83 $\pm$ 5.52	21.07 $\pm$ 7.03	52.03 $\pm$ 1.00	51.51 $\pm$ 1.20	55.14 $\pm$ 3.25	37.42 $\pm$ 3.07	69.96 $\pm$ 7.55	22.41 $\pm$ 16.55
SIGNET	60.84 $\pm$ 6.26	23.55 $\pm$ 3.33	62.28 $\pm$ 4.75	58.94 $\pm$ 5.24	49.80 $\pm$ 3.48	34.49 $\pm$ 3.17	61.94 $\pm$ 18.30	21.48 $\pm$ 16.91
iGAD	54.28 $\pm$ 6.26	84.16 $\pm$ 3.11	45.03 $\pm$ 3.65	55.38 $\pm$ 4.06	52.48 $\pm$ 2.29	68.55 $\pm$ 2.05	96.06 $\pm$ 0.97	99.57 $\pm$ 0.08
RQGNN	50.93 $\pm$ 4.37	82.19 $\pm$ 1.84	35.33 $\pm$ 2.06	42.75 $\pm$ 1.03	52.97 $\pm$ 1.34	68.19 $\pm$ 1.49	94.86 $\pm$ 3.17	99.39 $\pm$ 0.34
GPF	60.22 $\pm$ 2.10	89.00 $\pm$ 0.86	74.74 $\pm$ 1.40	79.98 $\pm$ 1.14	52.57 $\pm$ 1.50	69.40 $\pm$ 2.27	87.76 $\pm$ 9.53	98.88 $\pm$ 0.90
EdgePrompt	59.10 $\pm$ 2.61	88.74 $\pm$ 1.32	74.53 $\pm$ 2.60	79.50 $\pm$ 3.28	57.32 $\pm$ 1.24	72.89 $\pm$ 0.96	94.45 $\pm$ 2.27	99.51 $\pm$ 0.20
G-GLAD (ours)	62.17 $\pm$ 2.31	89.96 $\pm$ 0.80	76.95 $\pm$ 2.32	80.51 $\pm$ 2.07	59.45 $\pm$ 0.52	73.72 $\pm$ 0.42	96.61 $\pm$ 0.42	99.71 $\pm$ 0.04

Table 2: AUCs (% , mean $\pm$ std, 5 trials) of GLAD methods on dataset under the CAS setting. Methods, including our approach, classifier-based baselines, and graph prompt baselines, were trained with 15:1 and 7:1 normal-to-anomalous graph ratios.

Ratio	Method	AIDS	COX2	PROTEINS_full	DD	ENZYMES	IMDB-BINARY	REDDIT-BINARY	COLLAB
/	OCGIN	96.54 $\pm$ 2.01	60.30 $\pm$ 4.04	63.88 $\pm$ 5.75	71.40 $\pm$ 5.20	58.04 $\pm$ 7.57	64.59 $\pm$ 1.76	72.88 $\pm$ 3.22	70.94 $\pm$ 2.66
	OCGTL	96.70 $\pm$ 2.33	58.62 $\pm$ 5.61	62.12 $\pm$ 2.79	75.33 $\pm$ 3.74	57.20 $\pm$ 11.33	64.85 $\pm$ 2.36	76.11 $\pm$ 1.02	69.70 $\pm$ 0.87
	GLocalKD	95.24 $\pm$ 2.33	52.59 $\pm$ 3.59	75.13 $\pm$ 1.71	71.87 $\pm$ 2.91	54.83 $\pm$ 5.52	52.78 $\pm$ 1.72	78.27 $\pm$ 2.43	52.03 $\pm$ 1.00
	SIGNET	97.24 $\pm$ 1.17	71.27 $\pm$ 5.68	76.07 $\pm$ 3.54	70.33 $\pm$ 2.29	60.84 $\pm$ 6.26	67.44 $\pm$ 2.97	83.02 $\pm$ 1.26	62.28 $\pm$ 4.75
15:1	iGAD	71.04 $\pm$ 5.15	67.30 $\pm$ 1.30	68.82 $\pm$ 4.30	68.06 $\pm$ 2.19	52.29 $\pm$ 4.78	73.80 $\pm$ 3.43	OOM	74.88 $\pm$ 0.74
	RQGNN	96.68 $\pm$ 0.45	54.62 $\pm$ 1.47	75.70 $\pm$ 1.88	84.07 $\pm$ 0.67	59.97 $\pm$ 4.50	59.24 $\pm$ 0.80	79.94 $\pm$ 0.81	69.96 $\pm$ 8.73
	GPF	83.60 $\pm$ 1.24	64.78 $\pm$ 3.82	69.58 $\pm$ 1.40	66.34 $\pm$ 0.61	60.63 $\pm$ 2.00	76.79 $\pm$ 2.29	77.03 $\pm$ 1.02	71.47 $\pm$ 2.45
	EdgePrompt	86.33 $\pm$ 0.47	66.79 $\pm$ 5.40	67.36 $\pm$ 2.56	74.42 $\pm$ 0.90	58.81 $\pm$ 3.11	76.70 $\pm$ 1.84	79.38 $\pm$ 0.23	74.72 $\pm$ 2.59
	G-GLAD (ours)	99.97 $\pm$ 0.02	73.67 $\pm$ 3.80	78.55 $\pm$ 0.39	84.10 $\pm$ 0.56	62.85 $\pm$ 2.07	77.54 $\pm$ 1.42	83.45 $\pm$ 0.83	75.00 $\pm$ 3.02
7:1	iGAD	81.45 $\pm$ 4.64	72.29 $\pm$ 1.71	74.74 $\pm$ 0.98	69.16 $\pm$ 2.71	56.43 $\pm$ 6.39	76.95 $\pm$ 1.06	OOM	73.78 $\pm$ 1.75
	RQGNN	99.31 $\pm$ 0.34	68.25 $\pm$ 4.76	74.68 $\pm$ 1.82	82.85 $\pm$ 0.65	48.96 $\pm$ 9.08	64.63 $\pm$ 0.83	79.90 $\pm$ 0.83	73.17 $\pm$ 12.12
	GPF	83.58 $\pm$ 0.80	64.72 $\pm$ 3.10	69.72 $\pm$ 1.07	66.52 $\pm$ 0.80	61.16 $\pm$ 0.98	76.80 $\pm$ 2.31	77.05 $\pm$ 1.00	72.59 $\pm$ 2.25
	EdgePrompt	86.53 $\pm$ 0.42	66.25 $\pm$ 5.61	67.28 $\pm$ 2.57	74.09 $\pm$ 1.24	58.98 $\pm$ 2.86	76.70 $\pm$ 1.83	79.25 $\pm$ 0.41	74.69 $\pm$ 2.59
	G-GLAD (ours)	99.95 $\pm$ 0.03	74.51 $\pm$ 3.61	78.75 $\pm$ 0.44	83.74 $\pm$ 0.54	62.21 $\pm$ 2.28	77.01 $\pm$ 1.16	83.58 $\pm$ 0.72	74.81 $\pm$ 3.48

such methods may have limited ability to detect anomalies with different patterns. Although iGAD and RQGNN perform well under the CAS settings, their performance deteriorates significantly under OAS, revealing the limited generalization of methods that learn from classifiers. For example, comparing Table 1 and Table 2, the AUC of iGAD and RQGNN on COLLAB drops from 74.88% and 69.96% under CAS to 45.03% and 35.33% under OAS. By contrast, the AUC of GPF, EdgePrompt, and G-GLAD remains unchanged between CAS and OAS, demonstrating that graph prompt learning methods generalize well. Among them, G-GLAD outperforms both GPF and EdgePrompt. Notably, after removing the seen anomalies from the testing data, G-GLAD achieves the best performance at detecting unseen anomalies, as shown in Table 5 in Appendix A.4.1. These results indicate that the Graph Anomaly Prompter effectively expands the anomaly space, and when combined with the Subgraph-based Normality Extractor, it achieves better separation between normal and anomalous (seen and unseen) graphs.

5.2.2 Performance on Closed Anomaly-set Setting. From Table 2, we observe that G-GLAD consistently achieves the best detection performance across all datasets under the CAS setting. It produces higher and more stable AUCs than baselines that learn primarily from normal graphs, including OCGIN, OCGTL, GLocalKD, and

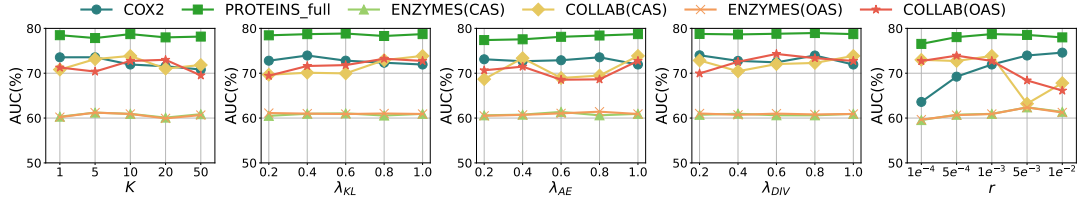
SIGNET. For classifier-based baselines such as iGAD and RQGNN, performance becomes unstable when the training set is more imbalanced, i.e., contains fewer anomalous samples. For example, on COX2, iGAD and RQGNN achieve 72.29% and 68.25% AUC with a 7:1 normal-to-anomalous ratio, but drop to 67.30% and 54.62% with a 15:1 ratio. Similar trends are observed on other datasets, indicating that these methods are sensitive to the proportion of anomaly samples in training. Moreover, due to the high spatial complexity of iGAD, processing REDDIT-BINARY requires over 80GB of memory, resulting in an out-of-memory (OOM) error. In contrast, the performance of G-GLAD, GPF, and EdgePrompt remains stable, indicating that classifier-based methods rely heavily on sufficient anomaly samples to model anomalies effectively. Furthermore, G-GLAD outperforms GPF and EdgePrompt, demonstrating the effectiveness of G-GLAD to differentiate anomalous graphs from normal ones.

5.3 Ablation Study

To investigate the contribution of each module in G-GLAD, we conduct an ablation study on four datasets, as reported in Table 3. First, we analyze the components within the Graph Anomaly Prompter. We consider three variants: removing the feature-level anomaly prompt (w/o FLAP), removing the structure-level anomaly prompt

Table 3: Ablation study on four datasets under the CAS and OAS setting w.r.t. AUC(%, mean $\pm$ std, 5 trials).

Dataset	COX2 (CAS)	PROTEINS_full (CAS)	ENZYMES (CAS)	COLLAB (CAS)	ENZYMES (OAS)	COLLAB (OAS)
w/o FLAP	71.82 $\pm$ 4.66	78.02 $\pm$ 0.89	61.99 $\pm$ 1.40	73.06 $\pm$ 2.96	61.32 $\pm$ 1.37	71.94 $\pm$ 1.39
w/o SLAP	71.45 $\pm$ 3.31	77.25 $\pm$ 1.49	61.02 $\pm$ 1.03	72.51 $\pm$ 2.77	61.01 $\pm$ 1.32	71.69 $\pm$ 2.56
w/o $\mathcal{L}_{DIV}$	72.16 $\pm$ 2.71	78.11 $\pm$ 0.28	61.58 $\pm$ 1.26	72.94 $\pm$ 2.10	60.94 $\pm$ 1.71	74.27 $\pm$ 4.03
w/o GAP	70.81 $\pm$ 0.83	76.22 $\pm$ 1.03	60.91 $\pm$ 1.45	70.51 $\pm$ 3.43	60.70 $\pm$ 0.61	71.67 $\pm$ 4.16
w/o SBNE	72.86 $\pm$ 2.75	77.40 $\pm$ 1.20	61.20 $\pm$ 0.26	69.71 $\pm$ 4.44	61.34 $\pm$ 0.77	69.21 $\pm$ 3.99
w/o both	69.61 $\pm$ 3.52	74.70 $\pm$ 1.48	60.04 $\pm$ 1.90	65.02 $\pm$ 6.72	59.35 $\pm$ 1.76	63.66 $\pm$ 4.36
G-GLAD	73.67 $\pm$ 3.80	78.55 $\pm$ 0.39	62.85 $\pm$ 2.07	75.00 $\pm$ 3.02	62.17 $\pm$ 2.31	76.95 $\pm$ 2.32

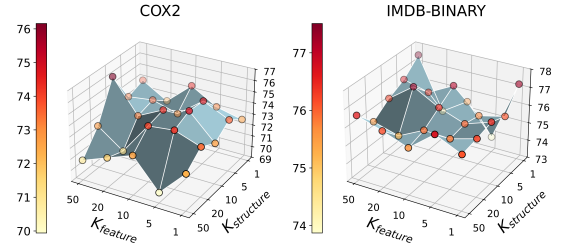
**Figure 2: Hyper-parameter sensitivity on K , λ_{KL} , λ_{AE} , λ_{DIV} and r w.r.t. AUC(%).**

(w/o SLAP), and removing the diversity loss (w/o $\mathcal{L}_{DIV}$). Notably, across all datasets, removing any of these components leads to a noticeable drop in AUC compared to the full G-GLAD. This demonstrates that each element—feature-level prompt, structure-level prompt, and diversity loss—contributes positively to G-GLAD. Next, we evaluate the overall contributions of the Graph Anomaly Prompter and the Subgraph-based Normality Extractor by considering three variants: removing Graph Anomaly Prompter (w/o GAP), removing Subgraph-based Normality Extractor (w/o SBNE), and removing both modules (w/o both). The results indicate that both modules consistently enhance performance. Notably, the w/o both variant shows the largest performance drop; for instance, on COLLAB (OAS), AUC decreases from 76.95% to 63.66%.

5.4 Hyper-parameter Sensibility

We investigate the impact of five hyper-parameters, including the number of anchor prompt vectors K for feature-level and structure-level prompts, with each level having its own set of K anchor vectors, the weighting coefficients λ_{KL} , λ_{AE} , λ_{DIV} , and the learning rate r . As shown in Figure 2, we make the following observations: $K = 1$ is generally suboptimal; G-GLAD is relatively insensitive to variations in λ_{KL} and λ_{DIV} ; a larger λ_{AE} can improve performance by expanding the anomaly space; and the optimal learning rate varies across datasets, though we recommend keeping it between 0.0005 and 0.005. Overall, G-GLAD maintains competitive performance across a wide range of hyper-parameter settings.

Furthermore, we examine the effect of varying the number of anchor prompt vectors at feature and structure levels independently. Specifically, we set $K_{feature}, K_{structure} \in \{1, 5, 10, 20, 50\}$. Figure 3 shows the performance on the COX2 and IMDB-BINARY datasets. For COX2, the best performance is achieved with $K_{feature} = 50$ and $K_{structure} = 10$, while for IMDB-BINARY, the highest AUC occurs at $K_{feature} = 50$ and $K_{structure} = 1$. This indicates that the optimal allocation of structure-level anchor prompts may vary across different types of graphs.

**Figure 3: Hyper-parameter sensitivity on $K_{feature}$ and $K_{structure}$ w.r.t. AUC(%).**

6 Conclusion

In this paper, we propose G-GLAD, a generalizable framework for graph-level anomaly detection. It comprises three complementary components: the Graph Anomaly Prompter, which injects learnable prompt vectors into graph attributes and structures to generate diverse anomalous variants; the Subgraph-based Normality Extractor, which leverages the information bottleneck principle to isolate minimal subgraphs sufficient to represent normality; and a Predictor that combines these representations to learn robust decision boundaries. Extensive experiments show that G-GLAD consistently outperforms state-of-the-art methods.

Acknowledgments

This work is supported by the Shanghai Pujiang Program (24PJJA004), the Fundamental Research Funds for the Central Universities (No. 25D111213), the National Natural Science Foundation of China (No. 92570118, No. 62376088, No. 62202095), the Hebei Natural Science Foundation (No. F2024202047), the Hebei Yanzhao Golden Platform Talent Gathering Programme Core Talent Project (Education Platform) (No. HJZD202509) and the Open Project Program of the State Key Laboratory of CAD&CG (No. A2411), Zhejiang University.

References

- [1] Xing Ai, Jialong Zhou, Yulin Zhu, Gaolei Li, Tomasz P Michalak, Xiapu Luo, and Kai Zhou. 2024. Graph anomaly detection at group level: A topology pattern enhanced unsupervised approach. In *Proceedings of the 2024 IEEE International Conference on Data Engineering*. 1213–1227.
- [2] Karsten M Borgwardt and Hans-Peter Krieger. 2005. Shortest-path kernels on graphs. In *Proceedings of the IEEE International Conference on Data Mining*. 1–8.
- [3] Jinyu Cai, Yunhe Zhang, and Jicong Fan. 2025. Self-Discriminative Modeling for Anomalous Graph Detection. In *Proceedings of the International Conference on Machine Learning*.
- [4] Shaofei Cai, Liang Li, Jincan Deng, Beichen Zhang, Zheng-Jun Zha, Li Su, and Qingming Huang. 2021. Rethinking graph neural architecture search from message-passing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6657–6666.
- [5] Nan Chen, Zemin Liu, Bryan Hooi, Bingsheng He, Rizal Fathony, Jun Hu, and Jia Chen. 2024. Consistency Training with Learnable Data Augmentation for Graph Anomaly Detection with Limited Supervision. In *Proceedings of the International Conference on Learning Representations*.
- [6] François Denis. 1998. PAC learning from positive statistical queries. In *Proceedings of the International Conference on Algorithmic Learning Theory*. 112–126.
- [7] Xiangyu Dong, Xingyi Zhang, and Sibao Wang. 2024. Rayleigh Quotient Graph Neural Networks for Graph-Level Anomaly Detection. In *Proceedings of the International Conference on Learning Representations*.
- [8] Taoran Fang, Yunchao Zhang, YANG YANG, Chunping Wang, and Lei Chen. 2023. Universal Prompt Tuning for Graph Neural Networks. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 36. 52464–52489.
- [9] Xingbo Fu, Yinhan He, and Jundong Li. 2025. Edge Prompt Tuning for Graph Neural Networks. In *Proceedings of the International Conference on Learning Representations*.
- [10] Dongxiao He, Yiqi Dong, Xiaobao Wang, Meng Ge, Carl Yang, Di Jin, and Witold Pedrycz. 2025. Unveiling Fine-grained Deceptive Patterns in Multi-modal Fake News: An Explainable Neuro-Symbolic Framework with LVLs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [11] Di Jin, Jingyi Cao, Xiaobao Wang, Bingdao Feng, Dongxiao He, Longbiao Wang, and Jianwu Dang. 2025. Rethinking contrastive learning in graph anomaly detection: a clean-view perspective. In *Proceedings of the International Joint Conference on Artificial Intelligence*.
- [12] U Kang, Hanghang Tong, and Jimeng Sun. 2012. Fast random walk graph kernel. In *Proceedings of the 2012 SIAM International Conference on Data Mining*. 828–838.
- [13] Nils M Kriege, Fredrik D Johansson, and Christopher Morris. 2020. A survey on graph kernels. *Applied Network Science* 5, 1 (2020), 6.
- [14] Jongha Lee, Sunwoo Kim, and Kijung Shin. 2024. Slade: Detecting dynamic anomalies in edge streams without labels via self-supervised learning. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1506–1517.
- [15] Xixun Lin, Yanan Cao, Nan Sun, Lixin Zou, Chuan Zhou, Peng Zhang, Shuai Zhang, Ge Zhang, and Jia Wu. 2025. Conformal graph-level out-of-distribution detection with adaptive data augmentation. In *Proceedings of the ACM on Web Conference 2025*. 4755–4765.
- [16] Xixun Lin, Yucheng Ning, Jingwen Zhang, Yan Dong, Yilong Liu, Yongxuan Wu, Xiaohua Qi, Nan Sun, Yanmin Shang, Kun Wang, et al. 2025. LLM-based Agents Suffer from Hallucinations: A Survey of Taxonomy, Methods, and Directions. *ArXiv Preprint ArXiv:2509.18970* (2025).
- [17] Yiqing Lin, Jianheng Tang, Chenyi Zi, H. Vicky Zhao, Yuan Yao, and Jia Li. 2024. UniGAD: Unifying Multi-level Graph Anomaly Detection. In *Proceedings of the Annual Conference on Neural Information Processing Systems*.
- [18] Yixin Liu, Kaize Ding, Qinghua Lu, Fuyi Li, Leo Yu Zhang, and Shirui Pan. 2023. Towards self-interpretable graph-level anomaly detection. *Advances in Neural Information Processing Systems* 36 (2023), 8975–8987.
- [19] Yixin Liu, Shiyuan Li, Yu Zheng, Qingfeng Chen, Chengqi Zhang, and Shirui Pan. 2024. ARC: A Generalist Graph Anomaly Detector with In-Context Learning. In *Proceedings of the Annual Conference on Neural Information Processing Systems*.
- [20] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM Web Conference 2023*. 417–428.
- [21] Dongsheng Luo, Wei Cheng, Wenchao Yu, Bo Zong, Jingchao Ni, Haifeng Chen, and Xiang Zhang. 2021. Learning to drop: Robust graph neural network via topological denoising. In *Proceedings of the ACM International Conference on Web Search and Data Mining*. 779–787.
- [22] Rongrong Ma, Guansong Pang, Ling Chen, and Anton Van Den Hengel. 2022. Deep graph-level anomaly detection by glocal knowledge distillation. In *Proceedings of the ACM International Conference on Web Search and Data Mining*. 704–714.
- [23] Xiaoxiao Ma, Jia Wu, Shan Xue, Jian Yang, Chuan Zhou, Quan Z Sheng, Hui Xiong, and Leman Akoglu. 2021. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2021), 12012–12038.
- [24] Siqi Miao, Mia Liu, and Pan Li. 2022. Interpretable and generalizable graph learning via stochastic attention mechanism. In *Proceedings of the International Conference on Machine Learning*. 15524–15543.
- [25] Hoang Nt and Takanori Maehara. 2019. Revisiting graph neural networks: All we have is low-pass filters. *ArXiv Preprint ArXiv:1905.09550* (2019).
- [26] Panagiotis Papadimitriou, Ali Dasdan, and Hector Garcia-Molina. 2010. Web graph similarity for anomaly detection. *Journal of Internet Services and Applications* 1, 1 (2010), 19–30.
- [27] Hezhe Qiao, Hanghang Tong, Bo An, Irwin King, Charu Aggarwal, and Guansong Pang. 2025. Deep graph anomaly detection: A survey and new perspectives. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [28] Chen Qiu, Marius Kloft, Stephan Mandt, and Maja Rudolph. 2022. Raising the Bar in Graph-level Anomaly Detection. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 2196–2203.
- [29] Jiawei Sheng, Taoyu Su, Weiye Yang, Linghui Wang, Yongxiu Xu, and Tingwen Liu. 2026. Information-Theoretic Minimal Sufficient Representation for Multi-Domain Knowledge Graph Completion. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [30] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. 2011. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research* 12, 9 (2011).
- [31] Chuancheng Song, Xixun Lin, Hanyang Shen, Yanmin Shang, and Yanan Cao. 2025. Uniform: Towards unified framework for anomaly detection on graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12559–12567.
- [32] Qingyun Sun, Jianxin Li, Hao Peng, Jia Wu, Xingcheng Fu, Cheng Ji, and Philip S. Yu. 2022. Graph Structure Learning with Variational Information Bottleneck. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 4165–4174.
- [33] Xiangguo Sun, Jiawen Zhang, Xixi Wu, Hong Cheng, Yun Xiong, and Jia Li. 2023. Graph prompt learning: A comprehensive survey and beyond. *ArXiv Preprint ArXiv:2311.16534* (2023).
- [34] Wenxuan Tu, Sihang Zhou, Xinwang Liu, Zhiping Cai, Yawei Zhao, Yue Liu, and Kunlun He. 2025. WAGE: Weight-Sharing Attribute-Missing Graph Autoencoder. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 7 (2025), 5760–5777.
- [35] Xiaobao Wang, Ruoxiao Sun, Yujun Zhang, Bingdao Feng, Dongxiao He, Luzhi Wang, and Di Jin. 2025. Stealthy Yet Effective: Distribution-Preserving Backdoor Attacks on Graph Classification. In *Proceedings of the Annual Conference on Neural Information Processing Systems*.
- [36] Zhengyang Wang and Shuiwang Ji. 2023. Second-Order Pooling for Graph Neural Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 6 (2023), 6870–6880.
- [37] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2020), 4–24.
- [38] Jun Xia, Lirong Wu, Jintao Chen, Bozhen Hu, and Stan Z Li. 2022. Simgrace: A simple framework for graph contrastive learning without data augmentation. In *Proceedings of the ACM Web Conference 2022*. 1070–1079.
- [39] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How Powerful are Graph Neural Networks. In *Proceedings of the International Conference on Learning Representations*.
- [40] Yiming Xu, Jiarun Chen, Zhen Peng, Zihan Chen, Qika Lin, Lan Ma, Bin Shi, and Bo Dong. 2025. Court of LLMs: Evidence-Augmented Generation via Multi-LLM Collaboration for Text-Attributed Graph Anomaly Detection. In *Proceedings of the ACM International Conference on Multimedia*.
- [41] Zhenyu Yang, Ge Zhang, Jia Wu, Jian Yang, Quan Z Sheng, Shan Xue, Chuan Zhou, Charu Aggarwal, Hao Peng, Wenbin Hu, et al. 2024. State of the art and potentialities of graph-level learning. *ACM Computing Surveys* 57, 2 (2024), 1–40.
- [42] Zhenyu Yang, Ge Zhang, Jia Wu, Jian Yang, Shan Xue, Amin Beheshti, Hao Peng, and Quan Z Sheng. 2025. Global Interpretable Graph-level Anomaly Detection via Prototype. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 3586–3597.
- [43] Xingtong Yu, Chang Zhou, Yuan Fang, and Xinming Zhang. 2024. Multigprompt for multi-task pre-training and prompting on graphs. In *Proceedings of the ACM Web Conference 2024*. 515–526.
- [44] Ge Zhang, Jia Wu, Jian Yang, Amin Beheshti, Shan Xue, Chuan Zhou, and Quan Z Sheng. 2021. Fraudre: Fraud detection dual-resistant to graph inconsistency and imbalance. In *Proceedings of the 2021 IEEE International Conference on Data Mining*. 867–876.
- [45] Ge Zhang, Zhenyu Yang, Jia Wu, Pengfei Jiao, and Jian Yang. 2025. Enhancing Graph Neural Networks for Out-of-Distribution Graph Detection. *IEEE Transactions on Neural Networks and Learning Systems* (2025), 1–15.
- [46] Ge Zhang, Zhenyu Yang, Jia Wu, Pengfei Jiao, Jian Yang, Hao Peng, and Xixun Lin. 2026. Learning Subgraph-Based Normality for Interpretable Graph-Level Anomaly Detection. *IEEE Transactions on Information Forensics and Security* 21 (2026), 288–303.
- [47] Ge Zhang, Zhenyu Yang, Jia Wu, Jian Yang, Shan Xue, Hao Peng, Jianlin Su, Chuan Zhou, Quan Z. Sheng, Leman Akoglu, and Charu C. Aggarwal. 2022. Dual-discriminative Graph Neural Network for Imbalanced Graph-level Anomaly Detection. In *Proceedings of the Advances in Neural Information Processing*

Systems.

- [48] Lingxiao Zhao and Leman Akoglu. 2023. On using classification datasets to evaluate graph outlier detection: Peculiar observations and new insights. *Big Data* 11, 3 (2023), 151–180.
- [49] Chenyi Zi, Haihong Zhao, Xiangguo Sun, Yiqing Lin, Hong Cheng, and Jia Li. 2024. ProG: A Graph Prompt Learning Benchmark. In *Proceedings of the Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

A Appendix

A.1 Pseudo-code Description

Algorithm 1 Training and inference procedures of G-GLAD.

```

1: #Training Procedure:
2: Input: training set  $\mathcal{D}_{tr}$ .
3: Output: model parameters.
4: Initialize the model parameters.
5: for each normal graph  $G^n \in \mathcal{D}_{tr}^n$  do
6:   Extract the subgraph  $G_S^n$  from  $G^n$  by Eq.(17).
7:   Obtain the representation  $\mathbf{h}_{G^n}$  of  $G^n$  by Eq.(1).
8:   Obtain the subgraph representation  $\mathbf{h}_{G_S^n}$  of  $G_S^n$  by Eq.(1).
9:   Calculate the KL-divergence by Eq.(18).
10: end for
11: for each anomalous graph  $G^a \in \mathcal{D}_{tr}^a$  do
12:   Obtain the representation  $\mathbf{h}_{G^a}$  of  $G^a$  by Eq.(1).
13:   Obtain representations  $\tilde{\mathbf{h}}_{G^a}^{\text{feature}}, \tilde{\mathbf{h}}_{G^a}^{\text{structure}}, \tilde{\mathbf{h}}_{G^a}^{\text{both}}$  of the
      prompted anomaly variants by Eq.(2-12).
14:   Calculate the anomaly expansion loss by Eq.(20).
15:   Calculate the diversity loss by Eq.(21-22).
16: end for
17: Calculate the cross-entropy loss by Eq.(19).
18: Calculate the final loss by Eq.(23).
19: Backpropagation.
20: #Inference Procedure:
21: Input: test dataset  $\mathcal{D}_{te}$  and model parameters.
22: Output: prediction results of  $\mathcal{D}_{te}$ .
23: for each test graph  $G_j \in \mathcal{D}_{te}$  do
24:   Generate its representation  $\mathbf{h}_{G_j}$  by Eq.(1).
25:   Calculate its detection results  $\hat{y}$  by predictor.
26:   if  $\hat{y} == 1$  then
27:      $G_j$  is an anomalous graph;
28:   else
29:      $G_j$  is a normal graph;
30:   end if
31: end for

```

Algorithm 1 summarizes the training and inference procedures of G-GLAD. The training dataset includes normal and anomalous graphs. Normal graphs are processed by the Subgraph-based Normality Extractor to obtain the key subgraphs that are essential for normality prediction, while anomalous graphs are processed by the Graph Anomaly Prompter to generate prompted anomaly variant representations. During inference, each test graph is encoded and classified by the predictor as normal or anomalous.

A.2 Variational Bounds for the optimization Objective – Eq. (14) and Eq. (16)

From Eq. (13), the optimization objective is:

$$\min_{\psi(\cdot)} I(G_S^n; G^n) - I(G_S^n; y), \quad \text{s.t. } G_S^n = \psi(G^n), \quad y = 0. \quad (24)$$

To optimize it, we introduce two variational bounds on the two terms, respectively. For the first term $I(G_S^n; G^n)$, by definition:

$$I(G_S^n; G^n) = I(G^n; G_S^n) = \mathbb{E}_{G^n, G_S^n} \left[\log \frac{P(G_S^n | G^n)}{P(G_S^n)} \right]. \quad (25)$$

Since $P(G_S^n)$ is intractable, we introduce a variational approximation $Q(G_S^n)$ for the marginal distribution:

$$P(G_S^n) = \sum_{G^n} P_{\psi}(G_S^n | G^n) P_{\mathcal{G}^n}(G^n). \quad (26)$$

The derivation proceeds as follows. First, the denominator is substituted with $Q(G_S^n)$, and a correction term is introduced. Then, expectations are taken on both sides concerning G^n and G_S^n . Owing to the non-negativity of the final term, its subtraction guarantees that the left-hand side is less than or equal to the previous term. Consequently, an upper bound for Eq. (16) is established as:

$$\begin{aligned} I(G_S^n; G^n) &= \mathbb{E}_{G^n, G_S^n} \left[\log \frac{P_{\psi}(G_S^n | G^n)}{P(G_S^n)} \right] \\ &= \mathbb{E}_{G^n, G_S^n} \left[\log \frac{P_{\psi}(G_S^n | G^n)}{Q(G_S^n)} + \log \frac{Q(G_S^n)}{P(G_S^n)} \right] \\ &= \mathbb{E}_{G^n, G_S^n} \left[\log \frac{P_{\psi}(G_S^n | G^n)}{Q(G_S^n)} \right] + \mathbb{E}_{G_S^n} \left[\log \frac{Q(G_S^n)}{P(G_S^n)} \right] \\ &= \mathbb{E}_{G^n} [\text{KL}(P_{\psi}(G_S^n | G^n) || Q(G_S^n))] - \text{KL}(P(G_S^n) || Q(G_S^n)) \\ &\leq \mathbb{E}_{G^n} [\text{KL}(P_{\psi}(G_S^n | G^n) || Q(G_S^n))]. \end{aligned} \quad (27)$$

For the second term $I(G_S^n; y)$, by definition:

$$I(G_S^n; y) = \mathbb{E}_{G_S^n, y} \left[\log \frac{P(y | G_S^n)}{P(y)} \right]. \quad (28)$$

The derivation of the lower bound proceeds as follows. To address the intractability of $P(y | G_S^n)$, we introduce a variational approximation $P_{\theta}(y | G_S^n)$. We begin by replacing the denominator with this approximation and incorporating a correction term. Subsequently, we take expectations with respect to G_S^n and y . As the last term is non-negative, we can derive a lower bound by omitting it. Finally, we arrive at the following lower bound for Eq. (14):

$$\begin{aligned} I(G_S^n; y) &= \mathbb{E}_{G_S^n, y} \left[\log \frac{P_{\theta}(y | G_S^n)}{P(y)} + \log \frac{P(y | G_S^n)}{P_{\theta}(y | G_S^n)} \right] \\ &= \mathbb{E}_{G_S^n, y} \left[\log \frac{P_{\theta}(y | G_S^n)}{P(y)} \right] + \mathbb{E}_{G_S^n} [\text{KL}(P(y | G_S^n) || P_{\theta}(y | G_S^n))] \\ &\geq \mathbb{E}_{G_S^n, y} \left[\log \frac{P_{\theta}(y | G_S^n)}{P(y)} \right] \\ &= \mathbb{E}_{G_S^n, y} [\log P_{\theta}(y | G_S^n)] - \mathbb{E}_y [\log P(y)] \\ &= \mathbb{E}_{G_S^n} [\log P(y = 0 | G_S^n)] + H(y = 0). \end{aligned} \quad (29)$$

Table 4: Detailed information of the graph benchmark datasets.

Dataset Name	# Graphs	# Average [V]	# Average [E]	# Node classes	# Graph Classes	Data Types
AIDS	2,000	15.69	16.2	38	2	Molecule
COX2	467	41.22	43.45	8	2	Molecule
PROTEINS_full	1,113	39.06	72.82	3	2	Biology
DD	1,178	284.32	715.66	82	2	Biology
ENZYMES	600	32.63	62.14	3	6	Biology
IMDB-BINARY	1,000	19.77	96.53	—	2	Social networks
IMDB-MULTI	1,500	13.00	65.94	—	3	Social networks
REDDIT-BINARY	2,000	429.63	497.75	—	2	Social networks
COLLAB	5,000	74.49	2457.78	—	3	Social networks
MSRC_9	221	40.58	97.94	10	8	Computer vision

Table 5: AUCs and AUPRs (% , mean $\pm$ std, 5 trials) of GLAD methods on dataset under OAS setting with seen anomalies excluded from the testing set. Methods, including our method, classifier-based baselines, and graph prompt baselines, were trained with a 15:1 normal-to-anomalous graph ratio. The best results are marked in bold.

Dataset	ENZYMES		COLLAB		IMDB-MULTI		MSRC_9	
Method	AUC	AUPR	AUC	AUPR	AUC	AUPR	AUC	AUPR
iGAD	51.90 $\pm$ 2.70	81.07 $\pm$ 1.21	49.65 $\pm$ 1.84	38.14 $\pm$ 1.55	48.93 $\pm$ 3.05	50.91 $\pm$ 3.84	46.45 $\pm$ 14.63	91.15 $\pm$ 2.75
RQGNN	56.36 $\pm$ 8.80	83.86 $\pm$ 5.00	38.94 $\pm$ 6.36	38.53 $\pm$ 6.32	53.70 $\pm$ 3.54	51.55 $\pm$ 2.91	90.91 $\pm$ 6.54	98.91 $\pm$ 0.92
GPF	60.10 $\pm$ 2.40	86.33 $\pm$ 1.69	52.87 $\pm$ 11.54	53.08 $\pm$ 12.93	56.06 $\pm$ 11.55	57.89 $\pm$ 9.62	92.88 $\pm$ 6.18	99.21 $\pm$ 0.68
EdgePrompt	58.38 $\pm$ 1.53	85.17 $\pm$ 1.26	75.96 $\pm$ 4.61	76.08 $\pm$ 2.40	65.16 $\pm$ 3.77	63.05 $\pm$ 3.73	94.38 $\pm$ 2.85	99.29 $\pm$ 0.31
G-GLAD (ours)	62.95 $\pm$ 0.61	87.63 $\pm$ 1.07	78.86 $\pm$ 4.22	76.35 $\pm$ 3.58	65.44 $\pm$ 0.97	63.97 $\pm$ 0.65	94.65 $\pm$ 1.23	99.40 $\pm$ 0.15

A.3 Detailed Experimental Setting

A.3.1 Dataset Details. Table 4 illustrates the detailed information of graph datasets used in our experiment. We adopt ten real-world datasets from four domains: (1) **Molecule datasets:** AIDS and COX2. They are small-molecule datasets, where nodes denote atoms and edges denote chemical bonds in molecular graphs. (2) **Biological datasets:** PROTEINS_full, DD, and ENZYMES. They are macromolecule datasets, where nodes usually represent secondary structure elements or amino acids, and edges represent that two nodes are sequence-neighbors or spatial neighbors as defined in the dataset. (3) **Social networks datasets:** IMDB-BINARY, IMDB-MULTI, REDDIT-BINARY and COLLAB. They are social networks datasets, where nodes generally denote people, actors and users etc., and edges denote interactions or co-occurrence. (4) **Computer vision dataset:** MSRC_9. It is a graph dataset derived from the Microsoft Research Cambridge object recognition image collection. Each graph corresponds to a single image that has been over-segmented into regions, nodes represent those regions and carry continuous attributes, while edges encode region adjacency. It is widely used as a benchmark to test methods’ sensitivity.

A.3.2 Metric Description. (1) **AUC** measures how well a model separates positive and negative classes across all possible decision thresholds by summarizing the ROC curve into a single scalar value. (2) **AUPR** integrates precision vs. recall tradeoffs for different thresholds, and is especially informative when positive and negative classes are imbalanced. (3) **Macro F1-score** computes the F1-score for each class independently then takes the unweighted average, giving equal importance to all classes regardless of their frequency.

A.4 More Experimental Results

A.4.1 Performance Comparison Exclusively on Unseen Anomalies under OAS Settings. Under the OAS setting, the AUC and AUPR

results for the models, trained on a 15:1 normal-to-anomalous ratio and tested on data with seen anomalies removed, are presented in Table 5. Across all datasets, G-GLAD achieves the best performance. Models like iGAD and RQGNN exhibit poor performance, while even graph prompt learning methods like GPF experience a significant performance degradation. For instance, on the COLLAB dataset: iGAD and RQGNN achieve low AUCs (49.65%, 38.94%) and AUPRs (38.14%, 38.53%), respectively. GPF attains an AUC of 52.87% and an AUPR of 53.08%. In stark contrast, G-GLAD demonstrates superior robustness with a much higher AUC (78.86%) and AUPR (76.35%). This represents a substantial performance gap; compared to G-GLAD, GPF’s AUC is approximately 33% lower and its AUPR is about 30.5% lower.

A.4.2 More Performance Comparison under CAS Settings. The comparison results under the CAS setting for AUPR, with training sets using 15:1 and 7:1 normal-to-anomalous ratios, are shown in Table 6. Similarly, the results for Macro F1-score under the same settings are reported in Table 7. Across most datasets, our proposed method G-GLAD achieves the best performance. The AUPRs and Macro F1-scores of G-GLAD remain stable at different ratios, further demonstrating the stability of G-GLAD. We can still see significant fluctuations in the performance of iGAD and RQGNN at different ratios. For instance, in Table 6, their AUPRs for COX2 achieve 47.80% and 39.60% at a 7:1 ratio, while they drop to 38.22% and 30.36% at a 15:1 ratio. In Table 7, their Macro F1-scores for AIDS achieve 64.63% and 93.83% at a 7:1 ratio, while they drop to 53.08% and 87.31% at a 15:1 ratio. This further confirms that classifier-based methods are sensitive to the proportion of anomaly samples in training.

A.4.3 Time Complexity. G-GLAD is trained and inferred in a batch-wise manner. The training time complexity is $O(L(|V| + |E|)d +$

Table 6: AUPRs (% , mean $\pm$ std, 5 trials) of GLAD methods on dataset under the CAS setting. Methods, including our approach, classifier-based baselines, and graph prompt baselines, were trained with 15:1 and 7:1 normal-to-anomalous graph ratios. The best results are marked in bold.

Ratio	Method	AIDS	COX2	PROTEINS_full	DD	ENZYMES	IMDB-BINARY	REDDIT-BINARY	COLLAB
/	OCGIN	97.14 $\pm$ 1.66	64.03 $\pm$ 4.14	53.57 $\pm$ 4.27	61.03 $\pm$ 4.59	88.25 $\pm$ 3.01	67.17 $\pm$ 1.21	74.87 $\pm$ 3.76	74.50 $\pm$ 2.20
	OCGTL	97.36 $\pm$ 1.79	62.92 $\pm$ 5.71	60.27 $\pm$ 1.81	66.20 $\pm$ 9.26	87.24 $\pm$ 4.85	67.45 $\pm$ 2.18	76.78 $\pm$ 1.26	73.19 $\pm$ 0.73
	GLocalKD	79.90 $\pm$ 6.25	78.46 $\pm$ 2.79	79.95 $\pm$ 1.89	76.00 $\pm$ 2.94	21.07 $\pm$ 7.03	54.85 $\pm$ 2.23	79.25 $\pm$ 2.55	51.51 $\pm$ 1.20
	SIGNET	80.93 $\pm$ 6.95	89.45 $\pm$ 2.18	81.76 $\pm$ 2.60	73.77 $\pm$ 2.99	23.55 $\pm$ 3.33	61.39 $\pm$ 2.08	82.61 $\pm$ 1.85	58.94 $\pm$ 5.24
15:1	iGAD	92.17 $\pm$ 1.48	38.22 $\pm$ 3.87	62.37 $\pm$ 2.62	62.56 $\pm$ 3.91	84.01 $\pm$ 2.65	75.56 $\pm$ 3.31	OOM	77.24 $\pm$ 0.86
	RQGNN	98.83 $\pm$ 0.07	30.36 $\pm$ 3.99	71.03 $\pm$ 1.80	79.09 $\pm$ 1.53	88.56 $\pm$ 1.52	63.80 $\pm$ 1.08	82.82 $\pm$ 1.46	70.84 $\pm$ 9.47
	GPF	95.98 $\pm$ 0.32	35.27 $\pm$ 3.00	63.01 $\pm$ 1.25	62.21 $\pm$ 0.93	89.08 $\pm$ 0.94	75.68 $\pm$ 2.24	79.05 $\pm$ 1.57	77.16 $\pm$ 2.44
	EdgePrompt	96.78 $\pm$ 0.10	38.07 $\pm$ 3.85	60.94 $\pm$ 2.44	69.68 $\pm$ 1.24	88.53 $\pm$ 1.50	75.38 $\pm$ 3.13	80.83 $\pm$ 0.71	77.54 $\pm$ 3.25
	G-GLAD (ours)	99.99 $\pm$ 0.00	55.27 $\pm$ 4.44	75.70 $\pm$ 0.09	79.24 $\pm$ 1.46	89.98 $\pm$ 0.66	75.78 $\pm$ 1.52	83.21 $\pm$ 0.51	78.20 $\pm$ 2.59
7:1	iGAD	95.37 $\pm$ 1.20	47.80 $\pm$ 2.52	66.71 $\pm$ 1.45	64.34 $\pm$ 3.90	86.08 $\pm$ 3.13	79.25 $\pm$ 1.23	OOM	77.63 $\pm$ 1.07
	RQGNN	99.80 $\pm$ 0.10	39.60 $\pm$ 6.03	72.59 $\pm$ 1.63	78.25 $\pm$ 1.26	84.49 $\pm$ 3.50	66.20 $\pm$ 1.95	82.69 $\pm$ 1.62	73.65 $\pm$ 12.25
	GPF	96.00 $\pm$ 0.17	35.89 $\pm$ 3.89	63.35 $\pm$ 1.59	62.62 $\pm$ 0.60	89.39 $\pm$ 0.70	76.68 $\pm$ 2.26	79.06 $\pm$ 1.59	77.30 $\pm$ 2.41
	EdgePrompt	96.85 $\pm$ 0.12	37.83 $\pm$ 4.59	61.04 $\pm$ 2.47	69.10 $\pm$ 1.92	88.69 $\pm$ 1.30	75.39 $\pm$ 3.12	81.25 $\pm$ 0.41	77.53 $\pm$ 3.26
	G-GLAD (ours)	99.99 $\pm$ 0.01	53.78 $\pm$ 3.86	75.58 $\pm$ 1.25	79.41 $\pm$ 0.58	90.02 $\pm$ 0.62	74.76 $\pm$ 2.54	83.24 $\pm$ 0.85	78.40 $\pm$ 3.47

Table 7: Macro F1-scores (% , mean $\pm$ std, 5 trials) of GLAD methods on dataset under the CAS setting.

Ratio	Method	AIDS	COX2	PROTEINS_full	DD	ENZYMES	IMDB-BINARY	REDDIT-BINARY	COLLAB
/	OCGIN	94.53 $\pm$ 2.10	53.64 $\pm$ 5.89	61.64 $\pm$ 4.96	66.10 $\pm$ 2.39	53.20 $\pm$ 4.49	58.79 $\pm$ 2.62	66.10 $\pm$ 3.02	64.42 $\pm$ 1.71
	OCGTL	94.38 $\pm$ 2.06	51.73 $\pm$ 4.21	59.40 $\pm$ 3.09	71.00 $\pm$ 3.19	54.40 $\pm$ 7.20	61.60 $\pm$ 1.50	70.90 $\pm$ 1.02	62.05 $\pm$ 1.22
	GLocalKD	88.76 $\pm$ 3.62	50.10 $\pm$ 4.52	67.52 $\pm$ 3.63	64.63 $\pm$ 4.26	39.37 $\pm$ 11.62	42.56 $\pm$ 7.80	75.26 $\pm$ 2.38	43.80 $\pm$ 1.78
	SIGNET	16.67 $\pm$ 0.00	57.19 $\pm$ 6.37	37.33 $\pm$ 0.03	36.97 $\pm$ 0.06	41.57 $\pm$ 5.05	33.33 $\pm$ 0.00	33.33 $\pm$ 0.00	34.21 $\pm$ 0.00
15:1	iGAD	53.08 $\pm$ 6.98	60.82 $\pm$ 4.23	66.85 $\pm$ 2.69	63.65 $\pm$ 3.11	28.88 $\pm$ 5.24	67.91 $\pm$ 1.97	OOM	69.02 $\pm$ 0.96
	RQGNN	87.31 $\pm$ 1.01	54.99 $\pm$ 1.60	68.20 $\pm$ 1.59	76.37 $\pm$ 0.77	52.83 $\pm$ 3.19	57.11 $\pm$ 1.60	75.05 $\pm$ 1.03	60.64 $\pm$ 15.61
	GPF	67.37 $\pm$ 1.99	45.07 $\pm$ 3.21	64.24 $\pm$ 1.28	61.66 $\pm$ 0.77	45.10 $\pm$ 5.53	68.73 $\pm$ 2.65	44.37 $\pm$ 1.54	49.39 $\pm$ 11.96
	EdgePrompt	62.38 $\pm$ 7.10	43.64 $\pm$ 0.00	61.49 $\pm$ 4.04	69.95 $\pm$ 0.70	51.51 $\pm$ 1.74	62.53 $\pm$ 11.54	68.86 $\pm$ 0.46	61.66 $\pm$ 1.60
	G-GLAD (ours)	99.63 $\pm$ 0.17	62.99 $\pm$ 2.93	70.41 $\pm$ 2.11	76.54 $\pm$ 0.67	59.35 $\pm$ 3.23	70.21 $\pm$ 0.81	75.33 $\pm$ 0.59	62.34 $\pm$ 5.26
7:1	iGAD	64.63 $\pm$ 2.63	63.10 $\pm$ 3.10	68.75 $\pm$ 1.46	64.07 $\pm$ 2.92	44.33 $\pm$ 7.99	69.99 $\pm$ 1.31	OOM	68.68 $\pm$ 0.94
	RQGNN	93.83 $\pm$ 2.83	57.43 $\pm$ 3.99	68.97 $\pm$ 0.80	73.79 $\pm$ 2.69	45.61 $\pm$ 3.26	61.99 $\pm$ 0.78	75.26 $\pm$ 0.72	66.24 $\pm$ 9.92
	GPF	65.91 $\pm$ 3.35	45.07 $\pm$ 3.21	64.59 $\pm$ 1.88	61.34 $\pm$ 1.17	46.51 $\pm$ 3.73	68.73 $\pm$ 2.65	44.41 $\pm$ 1.47	52.00 $\pm$ 5.22
	EdgePrompt	60.59 $\pm$ 7.78	43.64 $\pm$ 0.00	61.83 $\pm$ 4.29	69.37 $\pm$ 0.89	51.46 $\pm$ 1.70	62.53 $\pm$ 11.54	69.07 $\pm$ 1.56	61.65 $\pm$ 1.60
	G-GLAD (ours)	99.59 $\pm$ 0.00	63.16 $\pm$ 3.21	70.81 $\pm$ 2.31	76.90 $\pm$ 1.25	59.06 $\pm$ 2.31	70.06 $\pm$ 1.13	75.77 $\pm$ 0.35	60.38 $\pm$ 3.32

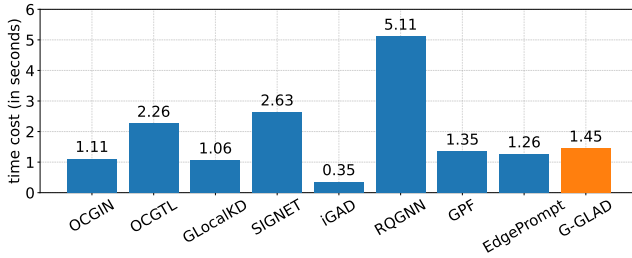


Figure 4: Training time comparison.

$L(|V_a| + |E_a|)Kd + |E_n|d^2 + |E_n| \log |E_n|$, where L is the number of GNN layers, d is the feature dimension, $|V|$ and $|E|$ are the numbers of nodes and edges, respectively. The first term corresponds to GNN encoding. The second term represents the complexity of Graph Anomaly Prompt generation, where K is the number of

prompt anchors, and $|V_a|$ and $|E_a|$ denote the total nodes and edges across all anomalous graphs. The last two terms account for the Subgraph-based Normality Extractor, where $|E_n|$ denotes the total edges belonging to normal graphs. During inference, the complexity reduces to $O(L(|V| + |E|)d)$.

We record the average time required by each algorithm to complete one epoch on the COLLAB dataset. We choose COLLAB because it contains the most densely connected graph structures among all datasets used in our experiments. For fairness, all competing methods are configured with identical hyper-parameter settings: a batch size of 64 and a hidden dimension of 32. The runtime comparison results are presented in Figure 4. As shown, G-GLAD achieves competitive runtime performance, while RQGNN requires substantially longer runtime, likely due to the high computational cost of graph spectral operations. Although iGAD has the shortest runtime, it demands substantial memory resources, leading to OOM issues on datasets such as REDDIT-BINARY.